Physical AI : Procedural Errors/Mistakes

Base Type(s)	
Status	Not started

Strategic Context: The Criticality of Procedural Integrity

Taxonomy of Procedural and Task-Sequence Errors

Step Omission

Step Addition

Sequence Misordering

Wrong Tool / Semantic Object Substitution

Early Termination & Incomplete Goal States

Step Correction / Recovery Loops

Strategic Context: The Criticality of Procedural Integrity

In the engineering of Physical AI, **Procedural Mistakes/Errors** also called **Structural Invalidity** is defined as a topological defect in a demonstration sequence where the state-space transitions deviate from the required task logic. These are not merely instances of stochastic noise in the demonstration manifold; they are manifestations of underlying cognitive failures that compromise the model's ability to learn deterministic causality.

Within this domain, procedural data integrity—specifically the logical consistency of robot demonstration sequences—serves as the foundational layer for high-performing Vision-Language-Action (VLA) policies.

The "So What?" factor is definitive: when structural invalidity is ingested into the training pipeline, it results in policy divergence during deployment. Specifically, if a demonstration sequence exhibits "Early Termination" (failing to return to a neutral "Home Position" or disengage a latch after a placement task), the resulting

VLA policy fails to tokenized the reset state. This leads to "hanging executors" in production, where the robot stalls or, worse, initiates high-risk physical collisions by attempting to execute a transition from an invalid prerequisite state. Procedural integrity is not a cleaning preference; it is an operational safety requirement.

Taxonomy of Procedural and Task-Sequence Errors

▼ Step Omission

1. General Definition

Step Omission occurs when an operator entirely skips or deletes a mandatory intermediate sub-task S_i from the prescribed operational sequence, transitioning directly from state s_{i-1} to s_{i+1} without establishing the physical or symbolic post-conditions of S_i.

Formal Mathematical & State-Graph Representation

- **Directed Acyclic Graph (DAG) State Transition**: In a task dependency graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V} = \mathcal{V}*S |cup |mathcal{V}A|) represents subgoals and atomic actions, and |(|mathcal{E}|) defines directed prerequisite edges:*
  $$|[|text{Expected Sequence: } A_1 |to A_2 |to |dots |to A{i-1} |to A_i |to A{i+1}\]$$
  $$\[\text{Observed Sequence: } A_1 \to A_2 \to \dots \to A_{i-1} \to A_{i+1}\]$$
- **Dynamic Programming Node-Dropping Penalty ($\lambda_{\text{drop_node}}$)**: In task-graph video alignment algorithms such as **GTG2Vid**, omitting step $u \in \mathcal{V}$ forces the dynamic programming cost matrix $\mathcal{D}(t, u)$ to drop node u from the path, incurring a localized node-dropping penalty $\lambda_{\text{drop_node}}$:
  $$\[\mathcal{D}(t, u) = \min_{p \in \text{Pred}(u)} \mathcal{D}(t, p) + \lambda_{\text{drop_node}}\]$$

- **Executable Set Violation ($\mathcal{E}_t$)**: In streaming task-tracking systems like **RegulAR**, an omission error is inferred at frame t whenever the predicted ongoing action $\hat{v}^A_t$ is not present in the currently unlocked executable set $\mathcal{E}_t$:
$$\hat{v}^A_t \notin \mathcal{E}_t \implies \text{Omission Error Flagged}$$

Cognitive & Human Factors Root Cause

Under cognitive human error frameworks (Reason's GEMS, THERP, SPAR-H, and HFACS-MEs), step omissions are primarily classified as:

- **Lapses (Memory & Attention Failure)**: Unintentional omissions caused by working memory overload, fatigue, or environmental interruptions. High mental workload during complex transitions causes the operator to falsely assume a prerequisite step was already completed.
- **Post-Completion Errors**: A specific cognitive lapse where an operator completes the main primary task (e.g., dispensing a reagent or assembling a part) and prematurely exits the workflow, omitting the secondary clean-up, un-clamping, or safety-reset sub-task.
- **Unsafe Acts & Preconditions (HFACS/STPA)**: Traced to level-2 preconditions (high task complexity, ambiguous work instructions, or cluttered person-machine interfaces) and level-3 supervisory failures (inadequate planning or uncalibrated pacing).

---

2. Subtypes & Atomic Classification

```
1. Step Omission
 ├── 1.1 Omission of Prerequisite Unlocking / Unsealing Action
 ├── 1.2 Omission of Verification, Inspection & Calibration
 ├── 1.3 Post-Completion Omission (Secondary Task Lapse)
 └── 1.4 Omission of Intermediate Process / Chemical / Thermal Reaction Step
```

1.1 Omission of Prerequisite Unlocking / Unsealing Action

- **Mechanism**: Attempting an interaction on a target component without performing the preliminary physical un-fastening, seal removal, or cover opening.

- **Physical Manifestation**: Gripper or operator hands approach an enclosed/locked vessel and apply forces directly to a sealed interface.

1.2 Omission of Verification, Inspection & Calibration

- **Mechanism**: Bypassing mandatory sensory or measurement checks (e.g., checking zero-load, verifying component orientation, or testing valve alignment) before initiating a joining/dispensing operation.
- **Physical Manifestation**: Direct contact with parts without prior pause or alignment fixture verification.

1.3 Post-Completion Omission (Secondary Task Lapse)

- **Mechanism**: Executing the core manipulation task but omitting required post-operational routines, such as capping a bottle, replacing a tool in its holder, or resetting safety interlocks.
- **Physical Manifestation**: Moving the end-effector away immediately after task completion while leaving the workspace in an unsealed or hazardous state.

1.4 Omission of Intermediate Process / Reaction Step

- **Mechanism**: Skipping an essential transforming step within a continuous sequence (e.g., omitting stirring, flux application, or pre-heating).
- **Physical Manifestation**: Transitioning directly from part positioning to part joining without the intermediate material conditioning step.

---

3. Domain-Specific Scenarios & Concrete Examples

Domain	Operational Context	Concrete Omission Scenario
Robotics & Teleoperation	Chemical/Reagent Liquid Handling	Operator attempts to aspirate liquid with a pipette without first removing the protective vial cap.
Robotics & Teleoperation	Mechanical Chassis Assembly	Operator attempts to drive an M6 bolt into a plate without first inserting the alignment dowel pin.

Domain	Operational Context	Concrete Omission Scenario
Medical & Surgical	Medication Administration (HFACS-MEs)	Healthcare provider administers an IV infusion without cross-checking the patient's allergy wristband (Dose/Check Omission).
Industrial Assembly	Pipework & Pressure Vessel Draining (PHECA)	Operator opens the drainage valve without verifying that the pressure isolation valve is closed.
Industrial Assembly	Motor Manufacturing (IndustReal)	Operator presses a bearing onto a shaft without applying the prescribed retaining compound/lubricant.
Egocentric Everyday	Culinary / Cooking (CaptainCook4D / EgoErrorVQA)	Preparing a sandwich by placing top bread directly onto mayo without adding the ham layer.
Egocentric Everyday	Equipment Setup (Epic-Tent)	Pitching a tent by raising the main poles without anchoring the ground stakes.

4. Edge Cases, Ambiguities & Boundary Conditions

A. Permissible Reordering vs. True Omission

In partially ordered task graphs (DAGs), two steps \(A\) and \(B\) may belong to the same topological tier (order-independent). If an operator executes \(B\) before \(A\), naive linear models flag \(A\) as "omitted" at time \(t\).

- **Boundary Rule**: A step is **only** classified as omitted if an attempted action \(C\) explicitly requires \(A\) as a structural prerequisite (\(\text{Pred}(C) \supset A\)) while \(A\) remains unexecuted. If \(A\) can still be validly executed later without invalidating \(B\), it is classified as a *Permissible Reordering*, not an omission.

B. Implicit / Off-Camera State Changes

An operator may receive components that were pre-cleaned, pre-cut, or pre-unsealed outside the camera's Field of View (FoV).

- **Boundary Rule**: Auto-QC engines must verify post-action state representations (via **Action Effect Modeling**) rather than relying solely on

visual motion tracking. If the initial visual frame shows the container already unsealed (`IsUnsealed = True`), the unsealing step is flagged as *Pre-Satisfied*, suppressing false omission alerts.

C. Conditional & Optional Steps

Steps that are executed only under specific environmental triggers (e.g., adding salt *if* liquid density is low).

- **Boundary Rule**: Conditional nodes in the task graph must evaluate sensory thresholds before declaring an omission. Skipping a conditional step when its trigger condition is false is a *Valid Bypass*.

---

5. Prerequisites & State Preconditions Violated

Formal PDDL Schema Assertion

For any action segment a_k, the symbolic state engine asserts that all required predicates in $\text{Preconditions}(a_k)$ are present in the current world state $S_{t_{\text{start}}}(a_k)$:

$$\text{Preconditions}(a_k) \subseteq S_{t_{\text{start}}}(a_k)$$

When a step omission occurs, this assertion fails:

$$\text{Preconditions}(a_k) \nsubseteq S_{t_{\text{start}}}(a_k) \implies \text{Deterministic Fault Flag Triggered}$$

Grounded Symbolic Predicate Examples

- **Container Manipulation**: $\text{Preconditions}(\text{Aspirate}(V)) = {\text{IsOpen}(V), \text{IsGrasped}(\text{Pipette})}$. If $\text{IsOpen}(V) = \text{False}$, step omission is flagged.
- **Fastener Assembly**: $\text{Preconditions}(\text{Tighten}(B)) = {\text{IsThreaded}(B), \text{IsAligned}(P_1, P_2)}$. If $\text{IsAligned} = \text{False}$, step omission is flagged.

---

6. Systemic & Policy Impact Analysis

```
Step Omission
 ├── Irreconcilable State-Action Mappings (a_t ~ π(s_{t+1}) on unconditioned s_{t-1})
 ├── Conflicting Gradients -> Inflated Action Divergence & Diffusion Score Matching Fa
ilure
 ├── Discrete Action Tokenizer Destabilization (FAST / π_0.5 out-of-distribution token
s)
 └── Downstream Physical Collisions, Mechanical Jams & Hardware Damage
```

A. VLA Policy & Diffusion Model Corruption

When demonstration datasets containing step omissions are ingested into Vision-Language-Action (VLA) architectures (e.g., OpenVLA, \(\pi_0\), ACT, or Diffusion Policy):

1. **Irreconcilable State-Action Mappings**: The model attempts to learn an action vector \(a_t \sim \pi(s_{t+1})\) conditioned on visual observations \(s_{t-1}\) where essential physical preconditions are missing.
2. **Gradient Conflict & Score Matching Failure**: The network receives conflicting loss gradients across visually similar frames (e.g., attempting to pour when the cap is on vs. off), inflating action divergence and causing score-matching instability in diffusion heads.
3. **Action Chunk Collapse**: Models utilizing continuous action chunking (ACT, Diffusion Policy) predict multi-step trajectory blocks \(H\). An omission within chunk horizon \(H\) forces the latent encoder to map discontinuous sub-task objectives into a single latent variable \(z\), causing chunk boundary discontinuities and severe joint chatter during deployment.
4. **Autoregressive Tokenizer Destabilization**: Architectures utilizing discrete action tokenizers (e.g., FAST in \(\pi_{0.5}\)) process control tokens alongside language tokens. Abrupt step omissions produce out-of-distribution token transitions, causing the autoregressive prior to collapse into erratic motor commands.

---

7. Detection, Recognition & Verification Methods

A. Offline Dataset Auto-QC Pipelines (Stage 3 Verification)

1. **Action Effect Modeling (AEM)**:
    - **Mechanism**: Uses **ActionFormer** to sample key effect frames immediately after predicted step completion ($t \in [ed_k, ed_k + \delta]$).
    - **Verification**: Extracts visual bounding boxes via **Grounding DINO** and builds symbolic scene graphs to verify physical state changes (e.g., checking if `Cap` node is detached from `Vial` node). If effect distance exceeds threshold τ_{effect}, omission is flagged.
2. **Generalized Task Graph Alignment (GTG2Vid & AMNAR)**:
    - **GTG2Vid**: Aligns untrimmed video with the task DAG using dynamic programming, penalizing skipped nodes via $\lambda_{\text{drop_node}}$.
    - **AMNAR**: Employs a Potential Action Prediction Block (PAPB) to filter segmentation noise via dynamic programming, and measures Euclidean distance between segment embeddings and valid successor prototypes ($d_{\min}(z_t) > \tau_k$).
3. **Neuro-Symbolic PDDL Verification Engine**:
    - Evaluates grounded state predicates prior to each action boundary, halting ingestion immediately upon precondition failure.

B. Online / Streaming Detection (RegulAR & Ego-ADR Frameworks)

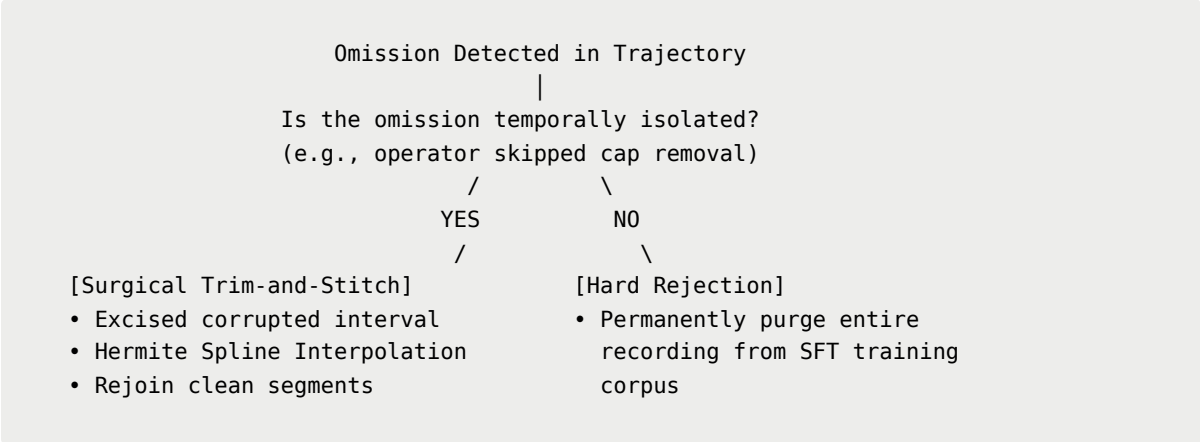
1. **RegulAR Executable Set Check**:
    - Tracks the set of executable actions $\mathcal{E}_t$ unlocked by satisfied DAG prerequisites. If a user attempts A_j while $A_j \notin \mathcal{E}_t$, an omission error is recognized instantly.
2. **Ego-ADR 3-Stage Decoupled VLM Reasoning**:
    - **Stage 1 (Key-step match)**: Localizes the attempted step within the procedure bank.
    - **Stage 2 (Video narration)**: Generates a neutral description of tools, actions, and scene state.

- **Stage 3 (Error classification)**: Compares narration against task preconditions to confirm omission with high precision, eliminating single-pass VLM hallucination.

---

8. Recovery Recommendations & Remediation Framework

A. For Offline Data Curation (Dataset Quality Control)



1. **Surgical Trim-and-Stitch**:
   - **Condition**: Applied when the step omission is temporally isolated and does not irreversibly corrupt downstream spatial geometry (e.g., operator hesitated, skipped a verification pause, and resumed).
   - **Procedure**: Excised the corrupted interval $t_{\text{start}} \to t_{\text{end}}$. Rejoin the preceding clean segment s_{pre} and succeeding clean segment s_{post} in proprioceptive space using third-order **Hermite Spline Interpolation** to guarantee C^1 velocity continuity.
2. **Hard Rejection**:
   - **Condition**: Applied when the omission permanently alters the physical environment (e.g., attempting to join parts without inserting a gasket, resulting in a physical crash).
   - **Procedure**: Purge the entire trajectory from the Golden SFT training set.

B. For Online Operator Guidance / AR Systems (RegulAR Schema)

When an omission is detected during live teleoperation or human assembly, the system delivers structured, **3-Part Intervention Feedback** anchored in topological impact scoring:

Impact Scoring Calculation (I_t)

The system calculates topological importance $\phi(a)$ based on downstream reachability $|\text{desc}(a)|$ and remaining task distance $\text{dist}(a, S_{\text{done}})$, multiplying it by skip-risk $\rho(a)$:
$$I_t = 1 - \prod_{a \in U_t} (1 - \phi(a) \cdot \rho(a))$$

Structured 3-Part Feedback Template

```
┌──────────────────────────────────────────────────────────────────────────┐
│ REGULAR AR INTERVENTION: STEP OMISSION DETECTED [Impact: HIGH (It = 0.82)]│
├──────────────────────────────────────────────────────────────────────────┤
│ 1. WHAT HAPPENED:                                                        │
│    "Prerequisite Step Skipped: You attempted to [Attempted Action, e.g., │
│    aspirate reagent] without completing [Skipped Step, e.g., unsealing   │
│    the vial cap]."                                                       │
│                                                                          │
│ 2. WHY IT MATTERS:                                                       │
│    "Impact Warning: Attempting this step now will cause mechanical       │
│    collision / fluid blockage and invalidates 4 downstream operations    │
│    [Subgoals S3, S4, S5]."                                               │
│                                                                          │
│ 3. HOW TO RECOVER:                                                       │
│    "Recovery Action: Pause current motion. Return to Step 1.2 [Remove    │
│    Vial Cap]. Once cap removal is detected, Step 2.1 will automatically  │
│    unlock."                                                              │
└──────────────────────────────────────────────────────────────────────────┘
```

🔍 **Next Steps**: Would you like to proceed with the detailed operational guideline for **2. Step Addition (Extraneous / Redundant Routines)** or **3. Step Misordering (Topological Inversions)** next?

▼ Step Addition

1. General Definition

Step Addition (also designated as *Extraneous Routine*, *Redundant Action*, or *Unintended Action*) occurs when an operator introduces unprescribed, task-irrelevant, or repetitive sub-routines into the execution pipeline. The system transitions from state s_t through one or more extraneous intermediate states $(s_{\text{extra},1}, \dots, s_{\text{extra},k})$ before returning to or advancing along the canonical task path, without achieving any required milestone or state predicate.

Formal Mathematical & State-Graph Representation

- **DAG State Path Violation**: Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be the Directed Acyclic Task Graph, where $\mathcal{V} = \mathcal{V}_S \cup \mathcal{V}_A$) represents valid subgoals and atomic actions. $$\text{Canonical Path: } P = (v_1, v_2, \dots, v_i, v_{i+1}, \dots, v_n)$$ $$\text{Observed Path: } P' = (v_1, v_2, \dots, v_i, v_{\text{extraneous}}, v_{i+1}, \dots, v_n) \quad \text{where } v_{\text{extraneous}} \notin \mathcal{V}_{\text{canonical}}$$
- **Dynamic Programming Frame-Dropping Penalty ($\lambda_{\text{drop_frame}}$)**: In task-graph alignment frameworks like **GTG2Vid**, when an operator performs an extraneous action or execution slip, advancing in time t without advancing to a new valid graph node u, the dynamic programming cost matrix $\mathcal{D}(t, u)$ activates a frame-dropping penalty $\lambda_{\text{drop_frame}}$: $$\mathcal{D}(t, u) = \mathcal{D}(t-1, u) + \lambda_{\text{drop_frame}}$$ This allows the alignment solver to isolate and skip extraneous frame blocks without losing global sequence alignment on the task graph.
- **Distance Thresholding in AMNAR**: In the **Adaptive Multiple Normal Action Representation (AMNAR)** framework, the Potential Action Prediction Block (PAPB) identifies all valid successor nodes $\text{Succ}(\mathcal{V}_{\text{executed}})$. *The Representation Matching Block (RMB) measures the minimum Euclidean distance between the ongoing segment feature* z_t *and all valid next action prototypes:*

$$d_{\min}(z_t) = \min_{k \in \text{Succ}(\mathcal{V}_{\text{executed}})} \|z_t - c_k\|_2$$
If $d_{\min}(z_t) > \tau_k$, the ongoing segment does not correspond to any valid successor step and is flagged as a **Step Addition / Unnecessary Action**.

Cognitive & Human Factors Root Cause

Under cognitive reliability and human error frameworks (Reason's GEMS, THERP, SPAR-H, HFACS-MEs, and STPA):

- **Skill-Based Slips / Habituation**: Automaticity and ingrained motor habits lead operators to execute familiar but unscripted sub-routines (e.g., compulsively wiping an already clean surface or shaking a tool).
- **Compulsive Re-Verification & Hesitation**: High mental workload or low confidence (elevated SPAR-H *Stress/Stressors* and *Task Complexity* PSFs) prompts the operator to perform repeated checking, re-measuring, or re-tightening.
- **Fiducial & Alignment Hunting**: Kinematic search behavior caused by poor ergonomics, ambiguous visual alignment marks, or unstable fixtures (HFACS Level 2: *Person-Machine Interface* and *Physical Environment preconditions*).
- **Operator Gaming & Protocol Violations**: Teleoperators introducing extraneous motions to pad active time metrics or manually reset master-slave arm kinematic drift.

---

2. Subtypes & Atomic Classification

```
2. Step Addition (Extraneous Routines)
 ├── 2.1 Fiducial & Orientation Hunting (Re-Grasping / Re-Seating)
 ├── 2.2 Unprescribed Auxiliary & Conditioning Routines
 ├── 2.3 Compulsive Re-Verification & Double-Checking
 └── 2.4 Unscripted Non-Markovian Recovery / Redo Loops
```

2.1 Fiducial & Orientation Hunting (Re-Grasping / Re-Seating)

- **Mechanism**: Repeatedly picking up, rotating, setting down, and re-grasping a component without making a functional state change.
- **Physical Manifestation**: In-air rotations, mid-air gripper adjustments, or sliding a part across a fixture before executing the actual insertion.

2.2 Unprescribed Auxiliary & Conditioning Routines

- **Mechanism**: Performing unscripted cleaning, wiping, shaking, or surface-tapping actions outside the operational protocol.
- **Physical Manifestation**: End-effector or operator hands executing repetitive oscillatory trajectories on workpiece surfaces or containers.

2.3 Compulsive Re-Verification & Double-Checking

- **Mechanism**: Re-measuring, re-weighing, or re-tightening a component or valve that has already reached the verified goal state.
- **Physical Manifestation**: Re-applying torque to a fully seated bolt or re-placing a container onto a digital scale after a completed reading.

2.4 Unscripted Non-Markovian Recovery Loops

- **Mechanism**: Attempting an unscripted, exploratory fix after a minor slip, introducing cyclic loops into the execution trajectory.
- **Physical Manifestation**: Disassembling and re-assembling a component that was already correctly engaged, creating a closed-loop trajectory cycle.

---

3. Domain-Specific Scenarios & Concrete Examples

Domain	Operational Context	Concrete Step Addition Scenario
Robotics & Teleoperation	Mechanical Assembly	Teleoperator rotates an M6 bolt 360° in mid-air multiple times while aligning the master-slave arm before insertion.
Robotics & Teleoperation	Reagent / Liquid Handling	Operator vigorously shakes a sealed reagent vial three times when the

Domain	Operational Context	Concrete Step Addition Scenario
		protocol specifies direct transfer without agitation.
Medical & Surgical (HFACS-MEs)	Clinical IV Administration	Nurse wipes an already sterile IV access port three separate unscripted times, introducing delays and potential contamination.
Industrial Assembly (IndustReal)	Motor & Bearing Assembly	Operator compulsively taps a fully seated bearing casing three times with a mallet after click-engagement is achieved.
Industrial Assembly (PHECA / COMAH)	Valve & Piping Maintenance	Technician re-applies a torque wrench to a locked valve flange four consecutive times after hearing the target click.
Egocentric Everyday (CaptainCook4D)	Culinary / Cooking	Operator stirs a soup mixture for an extra 45 unscripted seconds between slicing vegetables and adding seasoning.
Egocentric Everyday (EgoOops / Epic-Tent)	Equipment Setup	Operator repeatedly unrolls and re-rolls a guy-line that was already properly unrolled and ready for pegging.

4. Edge Cases, Ambiguities & Boundary Conditions

A. Adaptive Micro-Adjustments vs. True Step Addition

Closed-loop precision insertions (e.g., peg-in-hole or micro-connector seating) require subtle visual/force-guided micro-adjustments to align contact geometry.

- **Boundary Rule**: Trajectory adjustments are classified as **Valid Adaptive Micro-Adjustments** if the end-effector remains within the bounding volume of the active manipulation step and duration $t < \tau_{\text{micro}}$. If the motion disengages from the object, forms a distinct, self-contained sub-routine, or leaves the local workspace, it is classified as a **Step Addition**.

B. Protocol-Mandated Redundancy vs. Compulsive Re-Verification

High-risk industries (nuclear, aerospace, surgery) mandate formal double-checks (e.g., dual-operator sign-off or secondary torque verification).

- **Boundary Rule**: A re-verification step is **Valid** if it is explicitly represented as a node $v_{\text{check}} \in \mathcal{V}_{\text{task}}$ in the formal task graph. Unscripted, informal double-checking that exceeds prescribed node frequencies is classified as an **Unnecessary Step Addition**.

C. Demonstrator Style Variations vs. Unscripted Extraneous Actions

Human teleoperators naturally exhibit individual movement styles and speed variations.

- **Boundary Rule**: Style variation affects execution velocity and trajectory curvature within an action boundary. It is classified as an **Addition** only if it introduces an entirely new, recognizable action class or cyclic state transition outside the task DAG.

---

5. Prerequisites & State Preconditions Violated

Formal PDDL Schema Impact

In a Planning Domain Definition Language (PDDL) schema $\langle \text{Params}, \text{Preconditions}, \text{Effects} \rangle$, an action a_k should execute a state transition $S_t \xrightarrow{a_k} S_{t+1}$ where $\text{Effects}(a_k)$ modifies the world state toward the goal.

When an unnecessary step addition occurs:

1. **Null State Delta**: $S_t \xrightarrow{a_{\text{extraneous}}} S_t$ ($\Delta S = \emptyset$). The action consumes time T and computation without advancing any state predicate.
2. **Unwanted Side-Effect Predicates**: $S_t \xrightarrow{a_{\text{extraneous}}} S_{\text{corrupted}}$, introducing unwanted predicates such as $\text{IsSoiled}(\text{Part})$, \

(\text{IsDeformed}(\text{Gasket})\), or \(\text{IsHeatedPrematurely}(\text{Liquid})\).

Grounded Symbolic Predicate Example

- **Component Insertion**:
  \[\text{Preconditions}(a_{\text{insert}}) = {\text{IsGrasped}(P), \text{IsAligned}(P, H)}\]
  If an operator executes \(a_{\text{shake}} = \text{ShakeInAir}(P)\), \(\text{Effects}(a_{\text{shake}}) = \emptyset\). The action fails to establish \(\text{IsAligned}(P, H)\) and risks invalidating \(\text{IsGrasped}(P)\) via in-hand slip.

---

6. Systemic & Policy Impact Analysis

```
Step Addition (Extraneous Routine)
 ├── Injects Non-Markovian Loops (s_t -> s_extra -> s_t)
 ├── Traps Options Frameworks & Hierarchical Policies in Oscillatory Loops
 ├── Action Chunk Representation Collapse (Latent z averages trajectory jitter)
 └── Autoregressive Tokenizer Destabilization (FAST / π_0.5 out-of-distribution token
s)
```

A. Non-Markovian Loops & Policy Stalling

Extraneous routines introduce cyclic trajectories (\(s_t \to s_{\text{extra}} \to s_t\)) into demonstration datasets. When trained on these sequences, imitation learning policies (such as ACT or Diffusion Policy) learn non-Markovian state-action transitions, causing deployed robots to stall, oscillate between intermediate states, or enter infinite execution loops.

B. Action Chunk Representation Collapse

Architectures that predict multi-step action chunks over a temporal horizon \(H\) encode trajectory blocks into a latent variable \(z\). When an extraneous routine is embedded within chunk horizon \(H\), the encoder attempts to map discontinuous, irrelevant motion paths into \(z\), corrupting the latent space and producing severe joint chatter or erratic velocity spikes during deployment.

C. Discrete Action Tokenizer Destabilization

Foundation models utilizing discrete action tokenizers (e.g., FAST in \(\pi_{0.5}\) or \(\pi_0\)) interleave control tokens with text tokens. Unscripted additions introduce out-of-distribution control token sequences, causing the autoregressive prior to collapse and generate uncoordinated motor commands.

---

7. Detection, Recognition & Verification Methods

A. Offline Dataset Auto-QC Pipelines (Stage 3 Verification)

1. **GTG2Vid Dynamic Programming Alignment**:
    - Aligns untrimmed video frames against the task DAG. Extraneous frames trigger the frame-dropping penalty \(\lambda_{\text{drop_frame}}\), isolating the exact start and end timestamps of the redundant routine.
2. **AMNAR Potential Action Prediction**:
    - Uses PAPB and RMB to compare ongoing frame features \(z_t\) against all valid successor prototypes \(\text{Succ}(\mathcal{V}*{|text{executed}})|). If the segment distance |(d*{\min}(z_t) > \tau_k\), it is flagged as an off-graph step addition.
3. **Action Effect Modeling (AEM)**:
    - Evaluates post-action keyframe state changes. If ActionFormer detects an action boundary but the AEM effect token \(e_{\text{eff}}\) indicates zero state delta (\(\Delta S = 0\)), the segment is flagged as an extraneous motion.

B. Online / Streaming Detection (RegulAR & Ego-ADR Frameworks)

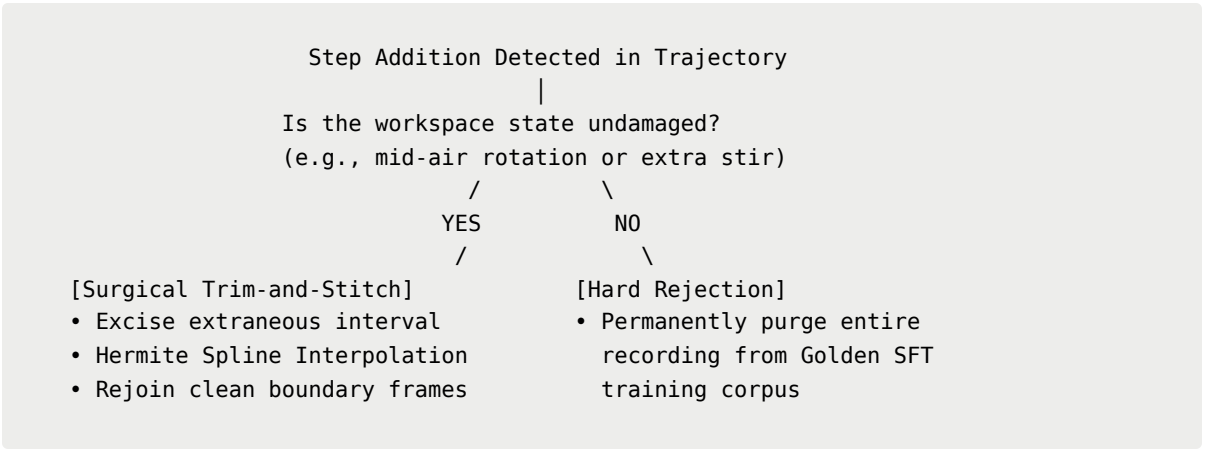
1. **RegulAR Graph Monitoring**:
    - Maintains an active executable set \(\mathcal{E}*t|). If an ongoing action |(|hat{v}^A_t|) is detected that is not present in the task graph |(|mathcal{V}*{\text{task}}\), RegulAR instantly flags an **Addition Error** and logs its impact score \(I_t\).

2. **Ego-ADR 3-Stage VLM Decoupled Reasoning**:
    - **Stage 1 (Key-Step Match)**: Attempts to match the ongoing action to the task procedure bank.
    - **Stage 2 (Video Narration)**: Generates a neutral description of the operator's movement.
    - **Stage 3 (Error Classification)**: Evaluates the narration against the procedure bank, recognizing that the action is unprescribed and classifying it as an **Unintended and Unnecessary Action** with high precision.

---

8. Recovery Recommendations & Remediation Framework

A. For Offline Data Curation (Dataset Quality Control)



1. **Surgical Trim-and-Stitch**:
    - **Condition**: Permissible when the extraneous routine does not alter or damage the physical workspace (e.g., mid-air part rotation, extra unscripted stirring, or temporary hesitation).
    - **Procedure**: Excise the extraneous temporal interval \((t_{\text{extra,start}} \to t_{\text{extra,end}})\). Rejoin the preceding clean frame \(s_{\text{pre}}\) and succeeding clean frame \(s_{\text{post}}\) in proprioceptive space using third-order **Hermite**

Spline Interpolation to ensure smooth velocity (\(C^1\)) and acceleration continuity.

2. **Hard Rejection**:
    - **Condition**: Mandatory if the extraneous action permanently damages a component, alters material properties (e.g., over-mixing a compound), or introduces unrecoverable spatial clutter.
    - **Procedure**: Purge the entire trajectory from the Golden SFT training corpus.

B. For Online Operator Guidance / AR Systems (RegulAR Schema)

When an unnecessary step addition is detected during live teleoperation or assembly, the AR system delivers structured **3-Part Intervention Feedback**:

Impact Scoring Calculation (\(I_t\))

Calculates the topological impact score based on added task duration, risk of component damage, and downstream reachability:
\[I_t = 1 - \prod_{a \in U_t} (1 - \phi(a) \cdot \rho(a))\]

Structured 3-Part Feedback Template

```
┌──────────────────────────────────────────────────────────────────────────┐
│ REGULAR AR INTERVENTION: STEP ADDITION DETECTED [Impact: MODERATE (It=0.54)]│
├──────────────────────────────────────────────────────────────────────────┤
│ 1. WHAT HAPPENED:                                                        │
│    "Unnecessary Action Detected: You performed an unscripted sub-routine │
│    [e.g., rotating the M6 bolt in mid-air / re-wiping the port]."        │
│                                                                          │
│ 2. WHY IT MATTERS:                                                       │
│    "Efficiency & Safety Warning: This action is not part of the task     │
│    graph. It adds unnecessary cycle time and risks in-hand component     │
│    slip or surface re-contamination."                                    │
│                                                                          │
│ 3. HOW TO RECOVER:                                                       │
│    "Recovery Action: Cease extraneous motion. Proceed directly to Step   │
```

```
│    3.2 [Insert M6 bolt into housing alignment hole]."                    │
└──────────────────────────────────────────────────────────────────────────┘
```

▼ Sequence Misordering

1. General Definition

Sequence Misordering (also designated as *Topological Inversion*, *Order Error*, or *Wrong Order*) occurs when an operator executes individually correct sub-tasks S_i in a temporal sequence that violates the governing Directed Acyclic Graph (DAG) dependencies or prerequisite state constraints. The operator attempts to transition from state s_A to s_C via action $a_{B \to C}$ before the prerequisite state s_B has been established by action $a_{A \to B}$.

Formal Mathematical & State-Graph Representation

- **DAG Prerequisite Violation**: Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ represent the task dependency graph, where directed edge $(v_u, v_v) \in \mathcal{E}$ dictates that action v_u is a strict prerequisite for action v_v.
  $$\text{Canonical Sequence: } P = (\dots, v_u, \dots, v_v, \dots) \quad \text{where } (v_u, v_v) \in \mathcal{E}$$
  $$\text{Observed Misordered Sequence: } P' = (\dots, v_v, \dots, v_u, \dots)$$
- **Procedure Order Similarity (POS) Metric**: In task alignment frameworks such as **IndustReal**, sequence adherence across an executed step sequence $\hat{\mathcal{P}}$ relative to ground-truth $\mathcal{P}$ is quantified using Damerau–Levenshtein distance without substitutions:
  $$\text{POS} = 1 - \min\left(\frac{\text{DamLev}(\mathcal{P}, \hat{\mathcal{P}})}{|\mathcal{P}|}, 1\right)$$
- **Transitive vs. Intransitive Error Propagation Rules**: As formalized in **Assembly101** action transition analysis, a misordered anchor step activates transition rules:

- *Transitive Rule*: Any subsequent step executed after an out-of-order anchor step inherits a propagated mistake flag because its physical preconditions are corrupted.
- *Intransitive Rule*: Only immediate child nodes directly dependent on the misordered step are flagged.

Cognitive & Human Factors Root Cause

Under cognitive reliability frameworks (Reason's GEMS, Rasmussen's SKK, SPAR-H, HFACS-MEs, and STPA):

- **Rule-Based & Knowledge-Based Mistakes**: Misinterpretation of complex work instructions or misapplication of task rules when operating under elevated cognitive load (SPAR-H *Task Complexity* and *Diagnosis* drivers).
- **Schema Capture & Action Inversions**: Execution of highly habituated motor schemas out of sequence due to peripheral distractions or reduced situational awareness (Rasmussen's Skill-Rule-Knowledge framework).
- **Preconditions for Unsafe Acts (HFACS Level 2)**: Multitasking, poor bin/part layout separation, and ambiguous visual markings leading directly to situational decision errors.

---

2. Subtypes & Atomic Classification

```
3. Sequence Misordering (Topological Inversions)
 ├── 3.1 Premature Action Execution (Inversion across Strict Dependent Nodes)
 ├── 3.2 Transitive Cascading Misordering (Propagated Error Chain)
 ├── 3.3 Inter-Step Transposition (Local Adjacent Swap)
 └── 3.4 Premature Verification / Out-of-Sequence Testing
```

3.1 Premature Action Execution (Strict Inversion)

- **Mechanism**: Executing a downstream assembly, pouring, or fastening action prior to completing its immediate, mandatory prerequisite node.
- **Physical Manifestation**: End-effector or operator hands bringing a secondary component to an unaligned or un-prepped workspace.

3.2 Transitive Cascading Misordering (Error Chain)

- **Mechanism**: Executing an early step out of order, causing all subsequent dependent actions to operate on an invalid physical state.
- **Physical Manifestation**: Continued assembly operations performed on top of a structurally inverted base component.

3.3 Inter-Step Transposition (Local Adjacent Swap)

- **Mechanism**: Swapping two adjacent steps S_i and S_{i+1} due to operator confusion or memory lapse.
- **Physical Manifestation**: Inverting the order of two closely coupled manipulation steps (e.g., placing top cover before inserting internal filter).

3.4 Premature Verification / Out-of-Sequence Testing

- **Mechanism**: Applying measurement gauges, pressure testing, or inspection probes before securing the underlying seal or joint.
- **Physical Manifestation**: Engaging diagnostic tools on incomplete physical assemblies.

---

3. Domain-Specific Scenarios & Concrete Examples

Domain	Operational Context	Concrete Sequence Misordering Scenario
Robotics & Teleoperation	Mechanical Assembly	Operator tightens a securing plate before inserting the internal alignment dowel pins.
Robotics & Teleoperation	Chemical & Beverage Prep	Operator pours hot water into a dripper assembly before placing the coffee grounds.
Medical & Surgical (HFACS-MEs)	Surgical & Clinical Prep	Healthcare provider applies surgical drapes before completing skin antiseptic scrubbing.
Industrial Assembly (IndustReal /	Motor Manufacturing	Operator presses a bearing casing onto a shaft before placing the

Domain	Operational Context	Concrete Sequence Misordering Scenario
Assembly101)		internal retaining ring.
Industrial Assembly (PHECA / COMAH)	Valve Maintenance	Technician applies sealing compound to flange threads *after* mating the pipe flange joints.
Egocentric Everyday (EgoErrorVQA / CaptainCook4D)	Culinary / Cooking	Operator closes a sandwich (placing top bread) before adding the ham and condiment layers.
Egocentric Everyday (Epic-Tent)	Equipment Setup	Operator attempts to raise the main tent support poles before threading them through the canvas sleeves.

4. Edge Cases, Ambiguities & Boundary Conditions

A. Permissible Order Variations vs. True Sequence Violations

In complex procedural task graphs (DAGs), two sub-tasks A and B may be topologically independent (order-independent parallel branches).

- **Boundary Rule**: Executing B before A is classified as a **Permissible Order Variation** if $(A, B) \notin \mathcal{E}$ and $(B, A) \notin \mathcal{E}$. It is classified as a **Sequence Misordering** *only* if a directed prerequisite edge $(A, B) \in \mathcal{E}$ exists in the task graph. Models like **AMNAR** prevent false positive alerts by dynamically generating normal action representations for all valid parallel child nodes.

B. Reversible Misordering vs. Irreversible Physical State Corruption

Some sequence inversions can be corrected non-destructively if caught immediately, whereas others cause permanent material damage.

- **Boundary Rule**: If a misordered action does not alter material properties or create permanent mechanical joins (e.g., placing an unfastened cover prematurely), it is classified as **Reversible Misordering**. If it involves permanent bonding, press-fitting without gaskets, or thermal/chemical reactions, it is classified as **Irreversible Physical Corruption**.

C. Disambiguating "Wrong Order" from "Wrong Action"

If an operator executes an action incorrectly *because* a preceding step was executed out of order, system taxonomies must distinguish the root cause.

- **Boundary Rule**: As defined in **EgoErrorVQA**, the initial inverted step is labeled **Wrong Order**. Subsequent dependent steps that fail due to propagated state corruption are classified as **Propagated Wrong Order** rather than independent "Wrong Action" errors.

---

5. Prerequisites & State Preconditions Violated

Formal PDDL Schema Assertion

In Planning Domain Definition Language (PDDL) syntax, every valid action is defined as:
$$\text{Action}(a_k) = \langle \text{Params}(a_k), \text{Preconditions}(a_k), \text{Effects}(a_k) \rangle$$

Prior to executing action segment a_k, the symbolic state engine asserts:
$$\text{Preconditions}(a_k) \subseteq S_{t_{\text{start}}}(a_k)$$

When sequence misordering occurs, this assertion fails:
$$\text{Preconditions}(a_k) \nsubseteq S_{t_{\text{start}}}(a_k) \implies \text{Deterministic Fault Flag Triggered}$$

Grounded Symbolic Predicate Example

- **Mechanical Assembly**:
  $$\text{Preconditions}(\text{TightenPlate}) = \{\text{IsInserted}(\text{DowelPin}), \text{IsSeated}(\text{Plate})\}$$
  If the operator executes TightenPlate while $\text{IsInserted}(\text{DowelPin}) = \text{False}$, the state engine flags a sequence misordering fault.

---

6. Systemic & Policy Impact Analysis

```
Sequence Misordering (Topological Inversion)
 ├── Irreconcilable State-Action Mappings (a_t ~ π(s_t) intended for s_B applied on s_
```

```
A)
 ├── Destabilizes Score Matching & Diffusion Head Training (Conflicting Gradients)
 ├── Destructive Physical Collisions, Joint Torque Spikes & Fixture Damage
 └── Action Tokenizer Collapse (FAST / π_0.5 out-of-distribution control tokens)
```

A. VLA Policy & Diffusion Model Corruption

When demonstration datasets containing sequence misorderings are ingested into Vision-Language-Action (VLA) foundation models (e.g., OpenVLA, \(\pi_0\), ACT, or Diffusion Policy):

1. **Irreconcilable State-Action Mappings**: The model attempts to learn an action vector \(a_t \sim \pi(s_t)\) intended to transition state \(s_B \to s_C\), while the visual observation is still in state \(s_A\).
2. **Diffusion Score Matching Failure**: The policy receives conflicting loss gradients across visually distinct environment states, corrupting score matching heads and inflating action divergence.
3. **Physical Collisions & Hardware Jams**: Executing manipulation actions without prerequisite seating during deployment leads directly to mechanical jams, bent pins, and gripper force-torque wrench spikes.
4. **Action Tokenizer Destabilization**: Architectures using discrete action tokenizers (e.g., FAST in \(\pi_{0.5}\)) receive out-of-sequence control tokens, causing autoregressive sequence generation to collapse into erratic motor commands.

---

7. Detection, Recognition & Verification Methods

A. Offline Dataset Auto-QC Pipelines (Stage 3 Verification)

1. **GTG2Vid Dynamic Programming Alignment**:
   - Aligns untrimmed video frame sequences against the task DAG. Out-of-order execution forces the dynamic programming cost matrix \(\mathcal{D}(t, u)\) to activate node-dropping penalties \(\lambda_{\text{drop_node}}\) for skipped prerequisites and frame-dropping penalties \(\lambda_{\text{drop_frame}}\) for premature steps.

2. **AMNAR Potential Action Prediction**:
    - Uses PAPB to build the longest common non-branching subsequence \(s_t^*\) on the task graph via dynamic programming. RMB then compares the ongoing action feature \(z_t\) against valid child node prototypes \(\text{Succ}(s_t^*)\). If distance \(d_{\min}(z_t) > \tau_k\), sequence misordering is flagged.
3. **Action Effect Modeling (AEM) & PDDL Verification**:
    - Evaluates post-action effect keyframes via **Grounding DINO** and VLM scene graphs. The PDDL state engine halts ingestion instantly if precondition predicates are unsatisfied.

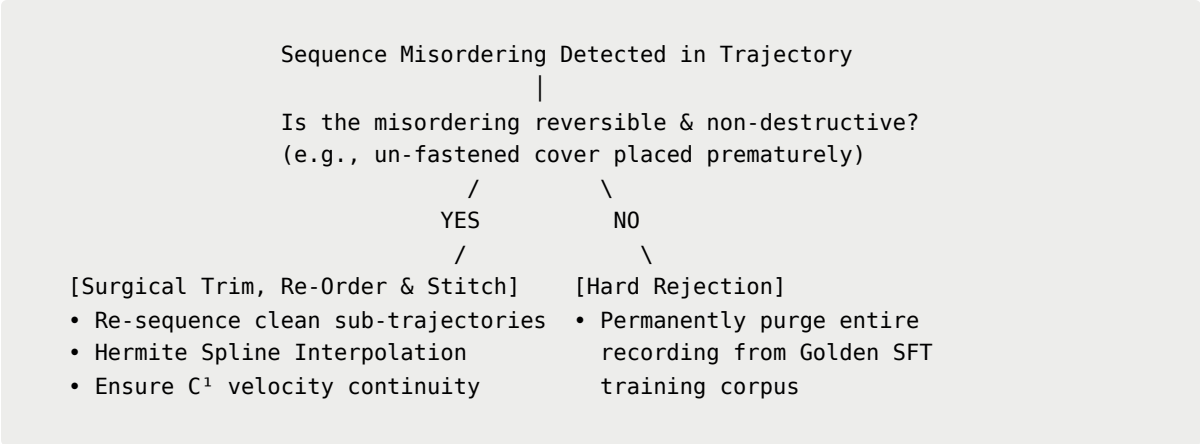
B. Online / Streaming Detection (RegulAR & Ego-ADR Frameworks)

1. **RegulAR Executable Set Check**:
    - Maintains an active executable set \(\mathcal{E}_t\) derived from satisfied DAG nodes. If an ongoing action \(\hat{v}^A_t \notin \mathcal{E}_t\), RegulAR recognizes a sequence violation, suppresses state oscillations, and calculates an impact score \(I_t\).
2. **Ego-ADR 3-Stage VLM Decoupled Reasoning**:
    - **Stage 1 (Key-Step Match)**: Localizes the current action within the procedure bank.
    - **Stage 2 (Video Narration)**: Generates a neutral description of current workspace state.
    - **Stage 3 (Error Classification)**: Compares narration against task preconditions, accurately classifying **Wrong Order** errors with high precision.

---

8. Recovery Recommendations & Remediation Framework

A. For Offline Data Curation (Dataset Quality Control)



1. **Surgical Trim, Re-Order & Stitch**:
    - **Condition**: Permissible *only* if the sequence inversion is physically reversible and caused no component deformation or permanent material bonding.
    - **Procedure**: Slice the recording into discrete sub-task clips. Re-arrange the sub-task clips into canonical DAG order. Rejoin adjacent segment boundaries in proprioceptive space using third-order **Hermite Spline Interpolation** to guarantee continuous position and velocity (\(C^1\)) profiles.
2. **Hard Rejection**:
    - **Condition**: Mandatory if the misordering caused physical collisions, joint torque spikes, or irreversible material changes.
    - **Procedure**: Purge the entire trajectory from the Golden SFT training corpus.

B. For Online Operator Guidance / AR Systems (RegulAR Schema)

When sequence misordering is detected during live teleoperation or assembly, the AR system delivers structured **3-Part Intervention Feedback**:

Impact Scoring Calculation (\(I_t\))

Calculates topological importance $\phi(a)$ based on downstream reachability $|\text{desc}(a)|$ and remaining task distance $\text{dist}(a, S_{\text{done}})$:
$$I_t = 1 - \prod_{a \in U_t} (1 - \phi(a) \cdot \rho(a))$$

Structured 3-Part Feedback Template

```
┌──────────────────────────────────────────────────────────────────────────┐
│ REGULAR AR INTERVENTION: SEQUENCE MISORDERING DETECTED [Impact: HIGH (It=0.78)]│
├──────────────────────────────────────────────────────────────────────────┤
│ 1. WHAT HAPPENED:                                                        │
│    "Sequence Misordering Detected: You attempted to execute [Attempted   │
│    Action, e.g., Tightening the securing plate] before completing        │
│    prerequisite [Missing Step, e.g., Inserting alignment dowel pins]."   │
│                                                                          │
│ 2. WHY IT MATTERS:                                                       │
│    "Critical Dependency Risk: Tightening the plate prematurely prevents  │
│    dowel pin insertion, blocking 3 downstream assembly steps [S4, S5, S6]│
│    and risking mechanical jam."                                          │
│                                                                          │
│ 3. HOW TO RECOVER:                                                       │
│    "Recovery Action: Pause tightening. Revert to Step 2.1 [Insert        │
│    Alignment Dowel Pins]. Once pin seating is detected, Step 2.2         │
│    [Tighten Securing Plate] will re-unlock."                             │
└──────────────────────────────────────────────────────────────────────────┘
```

▼ Wrong Tool / Semantic Object Substitution

1. General Definition

Wrong Tool / Semantic Object Substitution occurs when an operator or teleoperator executes a kinematically correct or visually plausible manipulation movement, but interacts with or applies an incorrect tool, component, or target object in the workspace. The action vector a_t is executed using object $O_{\text{actual}} \neq O_{\text{prescribed}}$, violating the task's semantic affordances and material requirements.

Formal Mathematical & Multimodal Representation

- **Semantic Role Labeling (SRL) Discrepancy**: Following the **MisFormer** and **MisEngine** attribution frameworks, an atomic step description S_i is

decomposed into a predicate (verb) P and a target argument (object) O:
$$\text{Instruction: } S_i = \langle P_i, O_{\text{target}} \rangle$$
$$\text{Observed Visual Execution: } \hat{S}*i = |langle P_i, O{\text{observed}} \rangle \quad \text{where } O_{\text{observed}} \neq O_{\text{target}}$$
In dual-branch multimodal architectures, cross-attending visual patch embeddings E_v with text role tokens $E_t(O_{\text{target}})$ produces a severe similarity drop:
$$\text{Sim}\left(E_v(O_{\text{observed}}), E_t(O_{\text{target}})\right) < \tau_{\text{semantic}} \implies \text{Semantic Substitution Flagged}$$

- **Zero-Shot Visual Grounding Loss**: In open-set object detection systems such as **Grounding DINO**, the vision backbone computes bounding box proposals B_k and feature representations f_k. Given text prompt $T = O_{\text{target}}$:
$$\max_k \text{IoU}\left(B_k, \text{GripBox}(t)\right) > 0.5 \quad \text{AND} \quad \text{CosineSim}(f_k, e(O_{\text{target}})) < \tau_{\text{ground}}$$
The end-effector is grasping an object whose spatial bounding box overlaps with active grip, but whose visual feature embedding deviates from the prescribed text token $e(O_{\text{target}})$.

Cognitive & Human Factors Root Cause

Under cognitive reliability and human error frameworks (Reason’s GEMS, THERP, SPAR-H, HFACS-MEs, and STPA):

- **Perceptual Lapses & Visual Similarity**: Misidentifying visually similar components (e.g., M4 vs. M6 bolts, or 10kΩ vs. 1kΩ resistors) due to poor lighting, specular reflections, or uncalibrated camera resolutions.
- **Workspace Clutter & Bin Contamination (HFACS Level 2 Preconditions)**: Poor physical layout or bin separation causing the operator to grab an adjacent wrong component from a mixed parts tray.
- **Habituation & Capture Slips**: Executing a motor schema toward a frequently used tool or component out of habit, ignoring the specific instruction prompt (Rasmussen's Skill-Rule-Knowledge framework).

- **Ambiguous Component Markings**: Insufficiently distinct color-coding, laser-etching, or part numbering on raw materials or medical vials.

---

2. Subtypes & Atomic Classification

```
4. Wrong Tool / Semantic Object Substitution
 ├── 4.1 Metric & Dimensional Mis-Selection (Geometry Mismatch)
 ├── 4.2 Functional Tool Substitution (Affordance Violation)
 ├── 4.3 Workspace Target Substitution (Clutter / Adjacent Bin Intrusion)
 └── 4.4 Material Grade & Chemical State Substitution
```

4.1 Metric & Dimensional Mis-Selection (Geometry Mismatch)

- **Mechanism**: Selecting a part of the correct object class but incorrect dimensional scale, thread pitch, or tolerance.
- **Physical Manifestation**: Grasping an M4 hex bolt instead of an M6 bolt, or an 8mm washer instead of a 10mm washer.

4.2 Functional Tool Substitution (Affordance Violation)

- **Mechanism**: Executing an action using an improper tool class that lacks the required physical force, torque limit, or end-effector geometry.
- **Physical Manifestation**: Using a flathead screwdriver on a Phillips fastener, or using a slicing knife instead of a spatula to spread adhesive/condiments.

4.3 Workspace Target Substitution (Clutter / Bin Intrusion)

- **Mechanism**: Reaching into an adjacent bin or picking up an unprescribed target object in a cluttered workspace due to spatial proximity.
- **Physical Manifestation**: Grasping a capacitor instead of an adjacent connector block on a dense PCB assembly bench.

4.4 Material Grade & Chemical State Substitution

- **Mechanism**: Utilizing a reagent, lubricant, or adhesive of incorrect concentration, viscosity, or chemical composition.

- **Physical Manifestation**: Dispensing high-viscosity threadlocker instead of low-viscosity retaining compound.

3. Domain-Specific Scenarios & Concrete Examples

Domain	Operational Context	Concrete Object/Tool Substitution Scenario
Robotics & Teleoperation	Chassis Assembly	Teleoperator grasps an M4×10mm bolt instead of the prescribed M6×15mm structural bolt.
Robotics & Teleoperation	Chemical Reagent Handling	Robotic arm picks up a vial containing \(0.1\text{M } \text{HCl}\) instead of \(1.0\text{M } \text{HCl}\) due to unread label text.
Medical & Surgical (HFACS-MEs)	Clinical Drug Delivery	Healthcare provider picks up a Heparin vial instead of a Saline flush vial from an cluttered tray.
Industrial Assembly (IndustReal)	Motor Housing Manufacturing	Operator selects a nylon-lock nut instead of a standard hex nut for temporary test jig clamping.
Industrial Assembly (PHECA / COMAH)	Flange Piping Maintenance	Technician uses a standard rubber O-ring instead of a high-temperature Viton gasket on a steam line.
Egocentric Everyday (CaptainCook4D / EgoErrorVQA)	Culinary / Cooking	Operator spreads hot mustard onto sandwich bread instead of mayonnaise.
Egocentric Everyday (EgoOops / Epic-Tent)	Equipment Setup	Operator attempts to secure a main tent ridge pole using a short awning peg instead of a heavy-duty ground stake.

4. Edge Cases, Ambiguities & Boundary Conditions

A. Permissible Functional Equivalents vs. True Substitution Errors

In manufacturing and repair, certain components are designated as valid substitutes (e.g., using a Grade 8.8 bolt when Grade 4.8 is specified, or using an equivalent cross-referenced tool).

- **Boundary Rule**: A substitution is classified as **Permissible Functional Equivalence** if the substitute item O_{sub} satisfies all PDDL load, dimension, and material state constraints ($\text{SatisfiesAllConstraints}(O_{\text{sub}}) = \text{True}$). It is classified as a **Semantic Substitution Error** if O_{sub} causes force/geometry mismatches or violates safety specs.

B. Workspace Clutter / Partial Occlusion vs. Object Error

An operator's hand or gripper may temporarily obscure the target object, causing 2D object detectors to output low-confidence predictions.

- **Boundary Rule**: Object classification must be evaluated at the **Point-of-No-Return (PNR)**—the exact frame where grasp closure or contact completes. If multi-view vision or post-grasp inspection confirms the correct object O_{target} despite partial occlusion during reach, it is classified as **Normal Execution with Occlusion**.

C. Temporary Auxiliary Tool Use

An operator may briefly use a tool to clear workspace clutter before grasping the prescribed assembly item.

- **Boundary Rule**: If the auxiliary tool interaction does not modify the primary component state and is followed by the correct object grasp, it is evaluated as an **Auxiliary Movement / Step Addition**, not a Wrong Tool Substitution.

---

5. Prerequisites & State Preconditions Violated

Formal PDDL Schema Assertion

In Planning Domain Definition Language (PDDL) syntax, an action definition explicitly binds target object types:

$$\text{Action}(a_k) = \langle \text{Params}(a_k): \{x \in \text{ToolType}, y \in \text{Workpiece}\}, \text{Preconditions}(a_k), \text{Effects}(a_k) \rangle$$

The symbolic state engine asserts that the bound parameters match required instance types:
\[\text{IsType}\left(\text{BoundObject}(a_k), \text{ExpectedType}(a_k)\right) = \text{True}\]

When a semantic object substitution occurs, this assertion fails:
\[\text{IsType}\left(O_{\text{actual}}, \text{ExpectedType}\right) = \text{False} \implies \text{Deterministic Semantic Fault Flag}\]

Grounded Symbolic Predicate Example

- **Threaded Fastening**:
  \[\text{Preconditions}(\text{DriveScrew}) = {\text{IsGrasped}(Tool), \text{IsType}(Tool, \text{PhillipsDriver}), \text{IsType}(Fastener, \text{M6Bolt})}\]
  If \(\text{IsGrasped}(Tool) = \text{FlatheadDriver}\), the precondition assertion fails, triggering an immediate fault flag.

---

6. Systemic & Policy Impact Analysis

```
Wrong Tool / Semantic Object Substitution
 ├── Cross-Attention Representation Corruption (Disjoint Visual vs. Prompt Tokens)
 ├── Policy Fine-Tuning Degradation (OpenVLA / π₀ learns conflicting embeddings)
 ├── Joint Wrench & Torque Spikes -> Thread Stripping, Tool Slippage & Part Damage
 └── Action Tokenizer Destabilization (FAST / π₀.₅ out-of-distribution control tokens)
```

A. VLA Policy Cross-Attention Alignment Corruption

In foundation Vision-Language-Action (VLA) models (such as OpenVLA, \(\pi_0\), or ACT), cross-attention layers align linguistic instruction tokens (e.g., *"Grasp M6 bolt"*) with visual patch embeddings.

1. **Multimodal Representation Mismatch**: When the visual patch depicts an M4 bolt (\(O_{\text{observed}}\)) while the paired instruction specifies an M6 bolt (\(O_{\text{target}}\)), the policy is forced to learn conflicting cross-attention weights.
2. **Degradation of Zero-Shot Generalization**: Training on mismatched pairs damages the model's latent vision-language grounding, causingDeployed

policies to fail at spatial localization and object selection in novel scenes.

3. **Physical Damage & Joint Torque Residuals**: Executing an M6 driving torque trajectory on an M4 bolt causes immediate thread stripping, tool cam-out, and severe 6-axis force-torque wrench spikes.

---

7. Detection, Recognition & Verification Methods

A. Offline Dataset Auto-QC Pipelines (Stage 3 Verification)

1. **Action Effect Modeling (AEM) & Grounding DINO**:
   - **Mechanism**: Uses **ActionFormer** to localize action boundaries. Immediately post-grasp, **Grounding DINO** detects the bounding box of the held item and evaluates open-vocabulary classification scores against task text prompts.
   - **Scene Graph Verification**: VLM scene graphs verify relational predicates (e.g., `Grasping(Gripper, M6_Bolt)` vs. `Grasping(Gripper, M4_Bolt)`). If confidence drops below threshold τ_{obj}, a substitution error is flagged.
2. **MisFormer Semantic Role Labeling (SRL) Cross-Attention**:
   - Decomposes step descriptions into Predicate and Object tokens. Dual-branch attention maps cross-attend role tokens against video features to isolate whether the error is a predicate violation or object substitution.

B. Online / Streaming Detection (RegulAR & Ego-ADR Frameworks)

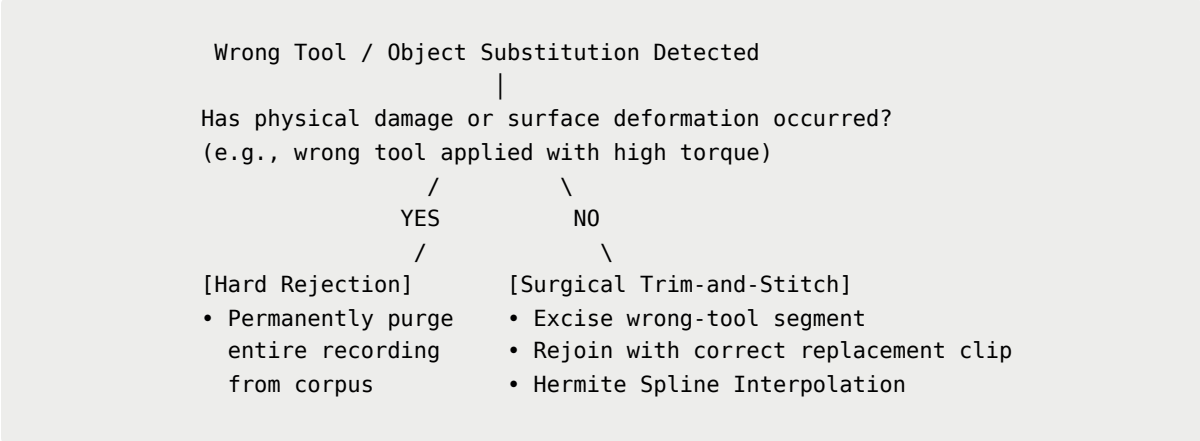
1. **RegulAR Graph & Metadata Check**:
   - Inspects egocentric video every 5 seconds. If the predicted held object $O_{\text{actual}} \neq O_{\text{graph_metadata}}$, RegulAR flags a **Modification / Substitution Error** and calculates an impact score I_t.
2. **Ego-ADR 3-Stage VLM Decoupled Reasoning**:

- **Stage 1 (Key-Step Match)**: Retrieves expected object and tool descriptions from the procedure bank.
- **Stage 2 (Video Narration)**: Generates a neutral description explicitly querying: *"What object or tool is the person actually holding?"*.
- **Stage 3 (Error Classification)**: Compares observed object text with expected object text, categorizing **Wrong Object / Wrong Tool** with high precision.

---

8. Recovery Recommendations & Remediation Framework

A. For Offline Data Curation (Dataset Quality Control)



1. **Hard Rejection**:
   - **Condition**: Mandatory if the wrong tool/object was driven with high force, causing thread stripping, component deformation, or surface contamination.
   - **Procedure**: Permanently purge the trajectory from the Golden SFT training set.
2. **Surgical Trim-and-Stitch**:
   - **Condition**: Permissible if the operator picked up the wrong item, recognized the mistake *before* applying force/joining, set it down, and picked up the correct item.

- **Procedure**: Excise the incorrect pickup-and-release interval \((t_{\text{wrong,start}} \to t_{\text{wrong,end}})\). Rejoin the clean approach and correct grasp frames in proprioceptive space using third-order **Hermite Spline Interpolation**.

B. For Online Operator Guidance / AR Systems (RegulAR Schema)

When a wrong tool or object substitution is recognized during live operation, the AR system delivers structured **3-Part Intervention Feedback**:

Impact Scoring Calculation (\(I_t\))

\[I_t = 1 - \prod_{a \in U_t} (1 - \phi(a) \cdot \rho(a))\]

Structured 3-Part Feedback Template

```
┌──────────────────────────────────────────────────────────────────────────┐
│ REGULAR AR INTERVENTION: WRONG OBJECT SUBSTITUTION [Impact: HIGH (It=0.81)]│
├──────────────────────────────────────────────────────────────────────────┤
│ 1. WHAT HAPPENED:                                                        │
│    "Wrong Object Detected: You grasped an [Observed Object, e.g., M4     │
│    Hex Bolt] instead of the prescribed [Target Object, e.g., M6 Structural│
│    Bolt]."                                                               │
│                                                                          │
│ 2. WHY IT MATTERS:                                                       │
│    "Damage Risk: Threading an M4 bolt into an M6 housing will cause      │
│    loose seating, joint instability, and invalidates downstream torque   │
│    verification [Step 4.3]."                                             │
│                                                                          │
│ 3. HOW TO RECOVER:                                                       │
│    "Recovery Action: Release the M4 bolt back into Bin B. Select the M6  │
│    bolt from Bin A (highlighted in Green HUD). Once M6 grasp is          │
│    confirmed, Step 4.1 will resume."                                     │
└──────────────────────────────────────────────────────────────────────────┘
```

▼ Early Termination & Incomplete Goal States

1. General Definition

Early Termination & Incomplete Goal States occurs when an operator or teleoperator halts, cuts, or aborts trajectory recording before the physical environment or target workpiece reaches the prescribed terminal goal state \(S_{\text{terminal}}\). The demonstration stops at an intermediate state \(s_{\text{premature}}\), failing to establish mandatory terminal post-conditions, spatial seating, or safety locking predicates.

Formal Mathematical & State-Graph Representation

- **Terminal Node Violation**: In a task dependency graph \(\mathcal{G} = (\mathcal{V}, \mathcal{E})\), let \(v_G \in \mathcal{V}*S|) represent the mandatory terminal goal node.*
  |[|text{Canonical Complete Path: } P = (v_1, v_2, |dots, v_n, v_G)|]
  |[|text{Observed Truncated Path: } P{\text{truncated}} = (v_1, v_2, \dots, v_{n-k}) \quad \text{where } k \ge 1 \text{ and } v_G \notin P_{\text{truncated}}\]
- **Action Effect Deficit (AEM State Distance)**: In **Action Effect Modeling (AEM)**, the final recorded frame \(I_{\text{final}}\) is projected into an effect-aware latent space via a learnable effect token \(e_{\text{eff}}\). The distance to the canonical goal prototype \(c_G\) is evaluated:
  \[d_{\text{goal}}(e_{\text{final}}) = |e_{\text{final}} - c_G|_2 > \tau_{\text{goal}} \implies \text{Incomplete Goal State Flagged}\]
- **Goal-Conditioned State Residual**: For a goal-conditioned policy \(\pi(a_t | s_t, g)\), the demonstration ends at \(s_{T_{\text{final}}}\) where the goal error metric exceeds threshold \(\epsilon_{\text{goal}}\):
  \[|f(s_{T_{\text{final}}}) - g| > \epsilon_{\text{goal}}\]

Cognitive & Human Factors Root Cause

Under cognitive reliability and human error frameworks (Reason’s GEMS, THERP, SPAR-H, HFACS-MEs, and STPA):

- **Premature Exit / Post-Completion Lapses**: The operator perceives the primary sub-action as "done enough" (e.g., part is placed in hole) and exits the recording before finalizing the secondary retention click, torque lock, or inspection pause.

- **Teleoperator Fatigue & Metric Gaming**: Rushing through demonstration logs to inflate per-hour trajectory throughput metrics, cutting recording buffers prematurely.
- **End-Trigger Execution Slips**: Accidental actuation of the stop-recording trigger or master-arm disengagement button prior to static stabilization.
- **Inadequate Work Instructions (HFACS Level 2 Preconditions)**: Ambiguous definition of what constitutes a valid "terminal state" in the operational brief.

---

2. Subtypes & Atomic Classification

```
5. Early Termination & Incomplete Goal States
 ├── 5.1 Premature End-Effector Release (Unseated Seating / Engagement)
 ├── 5.2 Truncated Dynamic Settling & Stabilization
 ├── 5.3 Omission of Terminal Safety Locking & Clean-Up
 └── 5.4 Partial Quantity / Volume Dispensing Truncation
```

5.1 Premature End-Effector Release (Unseated Seating / Engagement)

- **Mechanism**: Releasing the workpiece or tool before snap-fit engagement, thread seating, or magnetic latching completes.
- **Physical Manifestation**: Gripper opens while the component is hovering 1–5mm above the final resting shoulder or mating interface.

5.2 Truncated Dynamic Settling & Stabilization

- **Mechanism**: Stopping recording immediately upon releasing a part, without waiting for fluid sloshing, mechanical vibration, or sliding friction to settle to static equilibrium.
- **Physical Manifestation**: Video cuts while the target object is still oscillating, tilting, or sliding across the worksurface.

5.3 Omission of Terminal Safety Locking & Clean-Up

- **Mechanism**: Completing the primary assembly or dispensing action, but truncating the recording before re-engaging safety interlocks, capping containers, or returning tools to home docks.
- **Physical Manifestation**: Demonstration ends with open reagent bottles, exposed blades, or end-effectors left in active manipulation zones.

5.4 Partial Quantity / Volume Dispensing Truncation

- **Mechanism**: Aborting a filling, pouring, or heating step before reaching the prescribed volumetric, mass, or thermal setpoint.
- **Physical Manifestation**: Ending a dispense sequence at 80% volume capacity.

---

3. Domain-Specific Scenarios & Concrete Examples

Domain	Operational Context	Concrete Early Termination Scenario
Robotics & Teleoperation	Battery Pack Assembly	Teleoperator opens gripper and stops logging while a snap-fit battery cover is rested on the slot, before pressing down to achieve latch click.
Robotics & Teleoperation	Liquid Reagent Dispensing	Arm releases a pipette and ends recording while 0.8 mL has been transferred out of a prescribed 1.0 mL volume.
Medical & Surgical (HFACS-MEs)	Surgical Suturing	Surgeon places suture thread through tissue but cuts recording before tying and tightening the terminal securing knot.
Industrial Assembly (IndustReal)	Motor Bearing Pressing	Operator disengages press tool when bearing is seated 1.5mm above the mechanical shoulder stop.
Industrial Assembly (PHECA / COMAH)	Pressure Vessel Isolation	Technician closes an isolation valve to 85% travel and terminates the logging session before the mechanical stop is contacted.

Domain	Operational Context	Concrete Early Termination Scenario
Egocentric Everyday (CaptainCook4D)	Cooking / Culinary	Operator removes a baking tray from the oven before the timer expires or turns off heating element and leaves scene without capping oil.
Egocentric Everyday (Epic-Tent)	Equipment Setup	Operator inserts tent poles into corner grommets but ends video before tensioning the guy-lines.

4. Edge Cases, Ambiguities & Boundary Conditions

A. Intentional Mid-Trajectory Pause vs. Early Termination

Operators may pause movement to inspect workspace alignment before resuming.

- **Boundary Rule**: A recording is classified as an **Operator Pause** if velocity \(v \to 0\) for duration \(t\), but data logging continues until subsequent motion achieves \(S_{\text{terminal}}\). It is classified as **Early Termination** if the recording stream is explicitly severed before \(S_{\text{terminal}}\) is established.

B. Dynamic Settling Horizon (\(\tau_{\text{settle}}\))

In high-speed manipulation, elastic rebound or fluid sloshing requires time to reach rest.

- **Boundary Rule**: A trajectory must capture at least \(\tau_{\text{settle}} = 0.5\text{ s}\) of static equilibrium post-release. If recording terminates while visual optical flow or joint force sensors register active kinetic decay (\(E_{\text{kinetic}} > \epsilon\)), it is flagged as **Truncated Dynamic Settling**.

C. Multi-Stage Goal Dependencies

When a demonstration is designed to capture a single sub-task within a modular pipeline.

- **Boundary Rule**: Early termination must be evaluated against the **Sub-Goal Brief** g_k, not the global factory assembly goal G_{global}. If $s_{\text{final}} \models g_k$, the episode is valid.

---

5. Prerequisites & State Preconditions Violated

Formal PDDL Schema Assertion

In Planning Domain Definition Language (PDDL) syntax, a task domain includes a formal **Goal Specification Set**:

$$\text{TaskGoal}(T) = \{P_1, P_2, \dots, P_m\}$$

The symbolic state engine asserts that the final frame state S_{final} satisfies all goal predicates:

$$S_{\text{final}} \models \text{TaskGoal}(T)$$

When early termination occurs, this assertion fails:

$$\exists P_g \in \text{TaskGoal}(T) \quad \text{such that} \quad P_g \notin S_{\text{final}} \implies \text{Terminal Goal Deficit Flag}$$

Grounded Symbolic Predicate Example

- **Mechanical Snap-Fit**:
  $$\text{TaskGoal}(\text{BatteryCoverAssembly}) = \{\text{IsGrasped}(\text{Cover}) = \text{False}, \text{IsSeated}(\text{Cover}, \text{Slot}), \text{IsLatched}(\text{Cover})\times\}$$
  If S_{final} shows $\text{IsLatched}(\text{Cover}) = \text{False}$, the terminal goal assertion fails.

---

6. Systemic & Policy Impact Analysis

```
Early Termination & Incomplete Goal States
 ├── Goal-Conditioned Policy Collapse (π(a_t | s_t, g) fails terminal convergence)
 ├── Deployed Robot Stalls / Hovers 1cm Above Target Without Finalizing Contact
 ├── Diffusion Policy Vector Field Decay (Score matching collapses near boundary)
 └── FAST Tokenizer Freeze (Loss of terminal end-of-episode control tokens)
```

A. Goal-Conditioned VLA Policy Collapse

When goal-conditioned policies (such as π_0, OpenVLA, or ACT) are trained on demonstration datasets containing early terminations:

1. **Terminal Convergence Failure**: The policy $\pi(a_t | s_t, g)$ never sees training examples of final precision seating or contact stabilization. During deployment, as $s_t \to g$, the policy's predicted action vectors decay to zero prematurely, causing the robot gripper to hover 1cm above the target or stop right before snap-fit engagement.
2. **Score Matching Vector Collapse in Diffusion Policies**: Diffusion heads trained on truncated trajectories learn vector fields that lose gradient directional strength near terminal boundaries, causing deployed robots to stall or oscillate without completing the task.
3. **Discrete Tokenizer Freeze**: In models utilizing discrete action tokenizers (FAST in $\pi_{0.5}$), missing terminal end-of-episode tokens causes autoregressive generation to freeze in an uncompleted state.

---

7. Detection, Recognition & Verification Methods

A. Offline Dataset Auto-QC Pipelines (Stage 3 Verification)

1. **Action Effect Modeling (AEM) & Grounding DINO**:
   - **Mechanism**: AEM samples the final frame I_{final} of the trajectory. **Grounding DINO** and VLM scene graphs extract grounded relational predicates and measure feature distance against canonical goal prototypes c_G.
   - **Verification**: If $d(e_{\text{final}}, c_G) > \tau_{\text{goal}}$, or if scene graph verification shows missing predicates (e.g., `IsLatched = False`), an Early Termination error is flagged.
2. **PDDL Goal Engine**:
   - Evaluates grounded world state predicates on the terminal frame S_{final}, flagging an immediate fault if $S_{\text{final}} \not\models \text{TaskGoal}$.

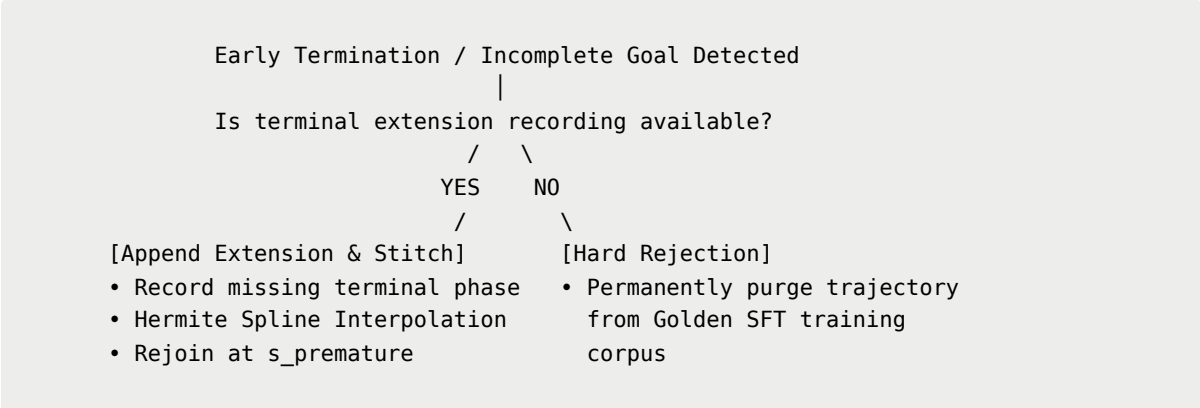
B. Online / Streaming Detection (RegulAR & Ego-ADR Frameworks)

1. **RegulAR Terminal Graph Check**:
    - Tracks DAG progression. If the operator attempts to trigger "Stop Recording" while unfulfilled terminal nodes $v_T \notin \mathcal{V}_{\text{completed}}$, RegulAR flags an **Incomplete Goal Error** and blocks file export.
2. **Ego-ADR 3-Stage VLM Decoupled Reasoning**:
    - **Stage 1 (Key-Step Match)**: Identifies expected terminal step.
    - **Stage 2 (Video Narration)**: Generates a neutral description of the final scene state.
    - **Stage 3 (Error Classification)**: Explicitly checks if terminal goal conditions are met, identifying **Early Termination / Incomplete Action** with high precision.

---

8. Recovery Recommendations & Remediation Framework

A. For Offline Data Curation (Dataset Quality Control)



1. **Hard Rejection (Default)**:
    - **Condition**: Mandatory for standard offline datasets where no extension recording is available. Because goal-conditioned VLA policies cannot

learn terminal convergence from incomplete goal states, truncated trajectories must **never** be included in the Golden SFT training set.

- **Procedure**: Purge the entire trajectory from the training corpus.

2. **Append Extension & Stitch (If Interactive)**:
   - **Condition**: Permissible only if the teleoperation system allows the operator to re-dock to state \(s_{\text{premature}}\) and record the missing terminal phase.
   - **Procedure**: Rejoin the original trajectory and extension clip in proprioceptive space using third-order **Hermite Spline Interpolation**.

B. For Online Operator Guidance / AR Systems (RegulAR Schema)

When early termination is attempted during live teleoperation or assembly, the AR system delivers structured **3-Part Intervention Feedback**:

Impact Scoring Calculation (\(I_t\))

Calculates maximum topological impact score because incomplete goal states invalidate the entire episode:
\[I_t = 1.0 \quad (\text{Critical Terminal Fault})\]

Structured 3-Part Feedback Template

```
┌──────────────────────────────────────────────────────────────────────────┐
│ REGULAR AR INTERVENTION: EARLY TERMINATION DETECTED [Impact: CRITICAL (It=1.0)]│
├──────────────────────────────────────────────────────────────────────────┤
│ 1. WHAT HAPPENED:                                                        │
│    "Premature Task Termination: You attempted to end recording before     │
│    completing terminal step [Missing Step, e.g., Snap-fit Cover Latch]."  │
│                                                                          │
│ 2. WHY IT MATTERS:                                                       │
│    "Critical Policy Failure: Saving an incomplete goal state prevents     │
│    robot policies from learning terminal convergence, causing deployed   │
│    robots to stall 1cm above targets."                                   │
│                                                                          │
│ 3. HOW TO RECOVER:                                                       │
│    "Recovery Action: Do not close session. Press down on the cover plate │
│    until the latch click is engaged (Step 5.2). Hold static for 0.5s     │
```

```
|    to complete goal verification."                                          |
└─────────────────────────────────────────────────────────────────────────────┘
```

▼ Step Correction / Recovery Loops

1. General Definition

Step Correction (also designated as *Corrective Actions*, *Step Recovery*, or *Mid-Task Redo Loops*) occurs when an operator or teleoperator recognizes an earlier execution error or state misalignment and introduces unscripted, unexpected recovery sub-routines—such as disassembling, re-grasping, re-aligning, or un-doing an action—to return the environment to a valid physical state before resuming task progression.

Formal Mathematical & State-Graph Representation

- **Cyclic State Transition Loop**: Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be the Directed Acyclic Task Graph. Instead of traversing a linear state path $s_t \to s_{t+1}$, a corrective loop introduces a cyclic transition trajectory:
  $$\text{Observed Path: } s_t \xrightarrow{a_{\text{error}}} s_{\text{fault}} \xrightarrow{a_{\text{undo}}} s_{\text{recovery}} \xrightarrow{a_{\text{re-align}}} s_t \xrightarrow{a_{\text{correct}}} s_{t+1}$$
  While the final state s_{t+1} reaches validity, the trajectory contains a closed-loop cycle $\mathcal{C} = (s_t \to s_{\text{fault}} \to s_{\text{recovery}} \to s_t)$ where the net state delta is zero ($\Delta S_{\text{net}} = \emptyset$).
- **Task-Graph Path Expansion Penalty**: In alignment solvers like **GTG2Vid**, corrective loops force the dynamic programming matrix $\mathcal{D}(t, u)$ to consume temporal frames t across backward or lateral graph edges, activating path-expansion and frame-dropping penalties ($\lambda_{\text{drop_frame}}$).
- **State Oscillation Detection**: In streaming runtime monitoring (e.g., **RegulAR**), a correction loop is recognized when the system observes repeated state transitions between an active node v_i and a previously completed node v_{i-k}, signaling an unscripted rollback.

Cognitive & Human Factors Root Cause

Under cognitive reliability and human error frameworks (Reason's GEMS, Rasmussen's SKK, SPAR-H, and HFACS-MEs):

- **Error Detection & Feedback Processing (GEMS Feedback Loop)**: Upon noticing a sensory discrepancy (e.g., visual misalignment, tactile resistance, or force-torque spike), the operator switches from automatic skill-based execution to conscious rule-based or knowledge-based problem-solving to undo the fault.
- **Elevated Task Complexity & High Stress (SPAR-H PSFs)**: Inadequate spatial feedback or poor teleoperation master-slave transparency causes minor slips that require continuous manual correction.
- **Unsafe Preconditions (HFACS Level 2)**: Slips during high-precision tasks prompt teleoperators to manually backtrack rather than aborting the recording, introducing unscripted recovery paths into dataset logs.

---

2. Subtypes & Atomic Classification

```
6. Step Correction / Recovery Loops
 ├── 6.1 Unscripted Disassembly & Backtracking (Undo / Demounting)
 ├── 6.2 Compensatory Realignment & Re-Grasping Loops
 ├── 6.3 Unprescribed Surface Clean-Up & Material Reset
 └── 6.4 Iterative Trial-and-Error Adjustment (Oscillatory Alignment)
```

6.1 Unscripted Disassembly & Backtracking (Undo / Demounting)

- **Mechanism**: Removing, unscrewing, or un-clamping a component that was just installed incorrectly, returning it to an earlier workspace location before re-attempting assembly.
- **Physical Manifestation**: End-effector reversing motion direction, disengaging fasteners, and extracting parts from assembly housings.

6.2 Compensatory Realignment & Re-Grasping Loops

- **Mechanism**: Releasing a misaligned or slipping component, opening gripper jaws, and executing a secondary grasp approach to improve grasp stability.
- **Physical Manifestation**: Gripper opens fully mid-task, hovers over workpiece, and re-closes on a revised contact centroid.

6.3 Unprescribed Surface Clean-Up & Material Reset

- **Mechanism**: Wiping away spilled liquid, scraping off excess adhesive, or clearing debris generated by a minor slip before re-applying the tool.
- **Physical Manifestation**: Operator hand or end-effector executing cleaning motions on work surfaces following an execution failure.

6.4 Iterative Trial-and-Error Adjustment (Oscillatory Alignment)

- **Mechanism**: Repeatedly pushing, nudging, or rotating a stubborn component across multiple failed insertion attempts until geometric alignment is achieved.
- **Physical Manifestation**: High-frequency oscillatory velocity profiles and fluctuating force-torque wrench signals centered around an assembly slot.

---

3. Domain-Specific Scenarios & Concrete Examples

Domain	Operational Context	Concrete Step Correction Scenario
Robotics & Teleoperation	Chassis Assembly	Teleoperator partially drives an M6 bolt, realizes it is cross-threaded, reverses the driver to extract the bolt, and re-threads it correctly.
Robotics & Teleoperation	Liquid Handling	Robotic arm knocks over a vial during transfer, pauses, rights the vial with its end-effector, wipes the surface, and resumes aspiration.
Medical & Surgical (HFACS-MEs)	Clinical IV Setup	Nurse inserts an IV catheter, notices backflow leakage, retracts the needle 1cm,

Domain	Operational Context	Concrete Step Correction Scenario
		re-adjusts angle, and re-advances the catheter.
Industrial Assembly (Assembly101)	Toy Car Assembly	Operator snaps a wheel onto the wrong axle, pulls the wheel off with force, and attaches it to the correct axle.
Industrial Assembly (IndustReal)	Motor Assembly	Operator places a bearing casing, observes a gap, uses a rubber mallet to tap it out, re-aligns the housing, and presses again.
Egocentric Everyday (CaptainCook4D)	Culinary / Cooking	Operator pours too much salt into a bowl, scoops out the excess with a spoon, and continues stirring.
Egocentric Everyday (Epic-Tent)	Equipment Setup	Operator inserts a tent pole into the wrong sleeve, pulls the pole back out completely, and threads it through the correct sleeve.

4. Edge Cases, Ambiguities & Boundary Conditions

A. Closed-Loop Adaptive Force Compliance vs. Unscripted Correction

High-precision robotic insertions use impedance/admittance control to adaptively adjust trajectories based on force feedback.

- **Boundary Rule**: Continuous, micro-scale trajectory adjustments executed within the active action boundary without disengaging contact or reversing macro-flow ($t < \tau_{\text{adjust}}$, displacement $< 5\text{mm}$) are classified as **Valid Adaptive Compliance**. If the motion involves an explicit disengagement, part extraction, or distinct "undo" phase, it is classified as a **Step Correction Loop**.

B. Planned Multi-Pass Operations vs. Corrective Redo Loops

Certain industrial processes mandate multi-pass routines (e.g., rough machining pass followed by finishing pass, or two-stage torqueing).

- **Boundary Rule**: Multi-pass steps are **Valid** if each pass is explicitly modeled as a separate node $v_{\text{pass1}}, v_{\text{pass2}} \in \mathcal{V}_{\text{task}}$ in the canonical task graph. Unscripted secondary passes executed in response to an unexpected defect are classified as **Step Correction Loops**.

---

5. Prerequisites & State Preconditions Violated

Formal PDDL Schema Assertion

In Planning Domain Definition Language (PDDL) syntax, a corrective loop introduces explicit "Undo" or "Reset" actions $\hat{a}_{\text{undo}}$ into the execution trace:

$$\text{Canonical Trace: } S_t \xrightarrow{a_k} S_{t+1}$$
$$\text{Observed Trace with Correction: } S_t \xrightarrow{a_k^{\text{fault}}} S_{\text{fault}} \xrightarrow{\hat{a}_{\text{undo}}} S_{\text{reset}} \xrightarrow{a_k^{\text{correct}}} S_{t+1}$$

The symbolic state engine asserts that state invariants and transition paths must remain monotonic toward the goal:
$$\text{IsMonotonicProgress}(S_0 \to S_{\text{final}}) = \text{False} \implies \text{Non-Markovian Correction Flagged}$$

---

6. Systemic & Policy Impact Analysis

```
Step Correction / Recovery Loop
 ├── Introduces Non-Markovian Cycles (s_t -> s_fault -> s_undo -> s_t)
 ├── Policy Mode-Splitting (VLA learns competing modes: Forward vs. Undo)
 ├── Deployed Robot Stalls or Enters Oscillatory "Do-Undo" Execution Loops
 └── Action Chunk Collapse (ACT / Diffusion Policy encodes discontinuous targets into
z)
```

A. VLA Policy Mode-Splitting & Oscillatory Stalling

When demonstration datasets containing unflagged corrective loops are ingested into Vision-Language-Action (VLA) foundation models (e.g., ACT, Diffusion Policy, OpenVLA, or π_0):

1. **Multimodal Mode Splitting**: The policy learns a bimodal state-action distribution for visual state s_{fault}: one mode predicting forward progress (a_{forward}) and another predicting unscripted undoing (a_{undo}).
2. **Infinite Execution Loops**: During deployment, when a minor trajectory drift occurs, the robot executes the learned undo motion (a_{undo}), returns to s_t, drifts again, and gets trapped in an infinite "do-undo" execution loop.
3. **Chunk Boundary Discontinuities**: Continuous action chunking models predict multi-step trajectory blocks H. Embedding a corrective reversal within chunk horizon H forces the latent encoder z to average opposing velocity vectors, producing severe joint chatter or mid-air freezing.

---

7. Detection, Recognition & Verification Methods

A. Offline Dataset Auto-QC Pipelines (Stage 3 Verification)

1. **GTG2Vid & AMNAR Graph Alignment**:
   - **GTG2Vid**: Detects backward state transitions on the task DAG, triggering node-dropping penalties for skipped steps and frame-dropping penalties for backward loops.
   - **AMNAR**: The Potential Action Prediction Block (PAPB) tracks the longest common valid subsequence s_t^*. When a correction loop occurs, segment features z_t deviate from valid child prototypes, triggering segment rejection flags.
2. **Action Effect Modeling (AEM) & ActionFormer**:
   - **ActionFormer** detects temporal boundaries of the fault and correction actions. **AEM** measures state deltas across effect frames ($I_{\text{fault}} \to I_{\text{undo}}$), confirming that an undo cycle occurred before valid post-conditions were re-established.

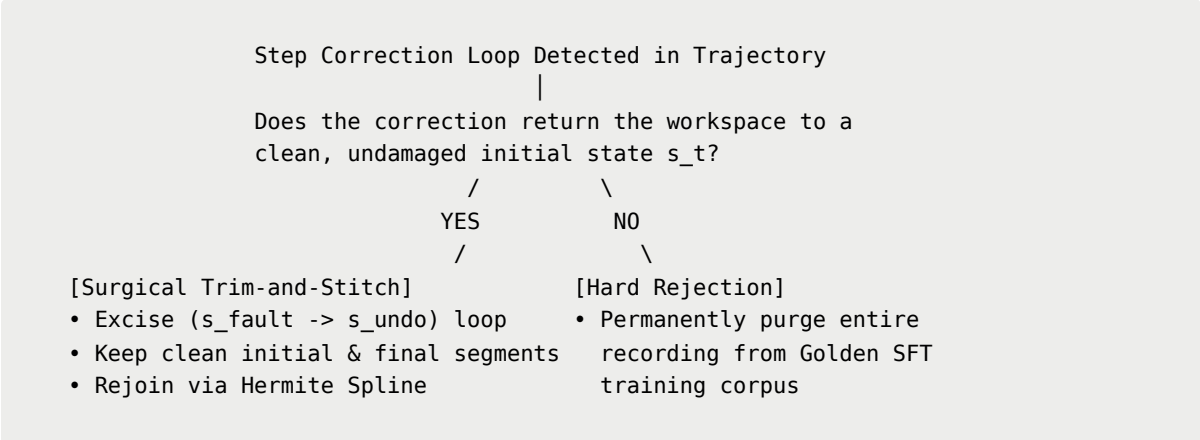
B. Online / Streaming Detection (RegulAR & Ego-ADR Frameworks)

1. **RegulAR State Oscillation Monitor**:
    - Tracks executable nodes \(\mathcal{E}*t*|*). If a user transitions backward to a previously completed node* |*(v*{\text{prev}}\), RegulAR flags a **Correction / Recovery Event** and monitors whether the recovery path reaches a valid graph node.
2. **Ego-ADR 3-Stage VLM Decoupled Reasoning**:
    - **Stage 1 (Key-Step Match)**: Identifies that the ongoing step corresponds to a previously completed or failed step.
    - **Stage 2 (Video Narration)**: Explicitly queries: *"Is the operator un-doing, re-adjusting, or fixing a prior action?"*.
    - **Stage 3 (Error Classification)**: Categorizes the event as a **Corrective Action / Recovery Loop** with high precision.

---

8. Recovery Recommendations & Remediation Framework

A. For Offline Data Curation (Dataset Quality Control)



1. **Surgical Trim-and-Stitch**:
    - **Condition**: Permissible if the correction loop cleanly restored the physical workspace without component deformation or surface

damage (e.g., re-grasping a dropped part or re-aligning an un-fastened cover).

- **Procedure**: Excise the corrupted interval $(t_{\text{fault,start}} \to t_{\text{undo,end}})$. Rejoin the clean initial frame s_{pre} and clean recovery frame s_{post} in proprioceptive space using third-order **Hermite Spline Interpolation** to guarantee continuous velocity (C^1) and acceleration (C^2) profiles.

2. **Hard Rejection**:
   - **Condition**: Mandatory if the initial fault caused thread stripping, surface contamination, or permanent component deformation during the redo attempt.
   - **Procedure**: Purge the entire recording from the Golden SFT training set.

B. For Online Operator Guidance / AR Systems (RegulAR Schema)

When a corrective recovery loop is detected during live operation, the AR system delivers structured **3-Part Intervention Feedback**:

Impact Scoring Calculation (I_t)

Calculates topological impact based on added cycle time and risk of secondary fault propagation:
$$I_t = 1 - \prod_{a \in U_t} (1 - \phi(a) \cdot \rho(a))$$

Structured 3-Part Feedback Template

```
┌──────────────────────────────────────────────────────────────────────────┐
| REGULAR AR INTERVENTION: RECOVERY LOOP DETECTED [Impact: MODERATE (It=0.62)]|
├──────────────────────────────────────────────────────────────────────────┤
| 1. WHAT HAPPENED:                                                        |
|    "Corrective Action Detected: You are un-doing / re-adjusting a prior  |
|    step [e.g., extracting cross-threaded M6 bolt]."                      |
|                                                                          |
| 2. WHY IT MATTERS:                                                       |
|    "Recovery Notice: Unscripted redo loops introduce non-Markovian cycles |
|    into dataset logs. Ensure the thread housing is clean before          |
```

```
|    re-threading."                                                          |
|                                                                            |
| 3. HOW TO RECOVER:                                                         |
|    "Recovery Action: Complete bolt extraction. Verify thread alignment     |
|    mark (Green HUD overlay). Re-insert bolt smoothly. Once seated, Step    |
|    3.1 will automatically resume."                                         |
└────────────────────────────────────────────────────────────────────────────┘
```